

Assignment - 6

M	T	W	T	F	S	S
Page No.						YOUVA
Date						

Expression conversion using Stack

Problem statement :

- Implement Stack as an ADT and apply it for different expression conversions (infix to postfix or infix to prefix (Any one), postfix to prefix (any one)).

Objective

- 1) To study stacks and its operations
- 2) To study the importance of expression conversions

Theory

- Stack : special case of ob-ordered list also called as restricted / controlled list where insertion and deletion happen at only one end called as top of stack (homogenous collection of elements). Elements are added to and removed from the top of the stack (The most recently added items are at the top of the stack). The last element to be added is the first to be removed (LIFO Last in, First out). Only access to the stack is the top element.

Basic Stack operations:

- Is Empty() - checking stack is empty
- Is full() - checking stack is full.

push () - Pushing element on top of stack

pop () - popping top element from stack.

• Applications of stack

1) Expression evaluation (postfix and prefix)

2) Infix to postfix and prefix conversion

3) Function call management (call stack)

4) Recursion

5) Parentheses Matching

6) Undo / Redo operations in editors

7) Browser back and forward navigation

8) Depth-first search (DFS) in graphs

9) Syntax parsing in compilers

10) String reversal

→ Expression conversion is the process of converting a arithmetic expression from one notation to another on a stack. The three common notations are:

- **Infix:** Operator is between operands (eg. $A+B$)

Prefix (Polish): operator is before operands (eg. $+AB$)

Postfix (Reverse Polish): operator is after operands
(eg. $AB+$)

A stack is used to temporarily store operators and parentheses during the conversion.

- **Implementation:**

Platform:

→ 64-bit open source linux on the derivatives

→ open source C programming tool like gcc/
Eclipse Editor

- **Text conditions:-**

1) Input infix expression and check the prefix or postfix output.

2) Input postfix or prefix expression and check its infix, postfix or postfix.

PSEUDO CODE

1

Push

Push (item)

If $top == max - 1$

Print "Stack overflow"

else

$top = top + 1$

$STACK[top] = item$

Pop()

If $top == -1$

Print "Stack Underflow"

else

$item = STACK[top]$

$top = top - 1$

Return item

• Infix to postfix

Create empty stack

for each symbol in expression

If symbol is an operand

Add symbol to postfix

Else if symbol is '('

PUSH (symbol)

Else if symbol is ')'

while top is not '('

Add Pop() to Postfix Pop '('

Pop '('

Else if symbol is an operator

while stack is not empty AND

precedence (TOP) $\geq$ precedence (symbol)

Add POP() to postfix

Push (symbol)

while stack is not empty

Add POP() to postfix

Return postfix

3) Time Complexity
 operation

time Complexity
 $O(1)$

1) push

$O(1)$

2) pop

$O(1)$

3) Is Empty

$O(1)$

4) Is full

$O(1)$

5) Infix to Postfix

$O(n)$

1) Prefix / postfix conversion

$O(n)$

→ The infix to postfix conversion is $O(n)$ because every character of the expression is processed a limited number of times.

• Space Complexity: $O(n)$, because the stack may store up to n elements.

FAQs

1) What is the advantage of prefix notation over infix expression?

Ans: Prefix and postfix expressions do not require parentheses to specify the order of operations.

Infix:

$$A + B * C$$

Postfix:

$$ABC^+$$

Prefix:

$$+A^*BC$$

The order of evaluation is directly determined by the position of the operators.

2) Why is stack used in expression conversion?

Ans: A stack follows LIFO (Last In, First Out) order. During expression conversion, operators and priorities need to be temporarily stored and stored processed according to their precedence and order.

Therefore, a stack is suitable for storing operators during conversion.

3) Give stack full and stack empty conditions.

A-5 For an array implementation of stack

Stack empty:

$$\text{Top} == -1$$

This condition is also called stack under overflow when we try to pop from an empty stack.

Stack Full:

$$\text{Top} == \text{max} - 1$$

This condition is called stack overflow when try to push into a full stack.